

How a Lean Repository Turns Formal Proofs into a Checked Public Release

Repository architecture, release flow, and trust boundaries

Will Cook

July 2026

Abstract

This document explains how one public mathematics repository is organised. The repository contains formal statements and proofs written in Lean, a program that checks proofs. It also contains papers, a front page, and other files that explain those proofs to people. Lean checks the formal mathematics; it does not read those explanations. The repository therefore adds a reviewed claim record between the proofs and the public documents. That record says what selected results mean, what evidence supports them, and what remains open. Programs build navigation files from the record and stop a release when named relationships no longer agree. GitHub runs the formal proof check and the public-release checks as separate jobs. A mathematician still decides whether the formal statement is the intended one and whether the public words describe it faithfully. The architecture makes those reviewed decisions repeatable; it does not replace them. This paper names the actual files, shows the complete flow, follows one result from Lean source to a public page, and states exactly where the automatic guarantee ends.

1 What this repository is for

This is a public collection of files about two unsolved problems in mathematics, Erdős Problems 249 and 257. Both problems remain open. The project proves intermediate results, exact reformulations, and finite examples around them. It does not claim to solve either problem.

The collection has two jobs:

1. It stores precise mathematical statements and proofs in *Lean*. Lean is a proof assistant: a program that checks whether a formal proof establishes the exact formal statement written for it.
2. It explains that work to people through a README, a scope statement, papers, source maps, and data that other programs can read.

A *repository* is simply the versioned collection containing those files. A *public claim* is a statement to readers about what the project has proved or has not proved. A *release* is one fixed version of the files offered to readers. No knowledge of Lean, Git, number theory, or the project's development history is assumed in this document.

The central problem is easy to state. A Lean proof can remain correct while a paper or README describes it incorrectly. For example, a proof covering a finite list of inputs can be described accidentally as a result covering all inputs. Lean cannot catch that wording error because the README is not part of the formal proof.

The repository addresses this gap with five visible parts:

1. **Lean source** containing formal statements and proofs;

2. a **human-reviewed claim record** describing selected results and their limits;
3. **authored public documents** such as the README and papers;
4. **generated indexes and summaries** that reorganise recorded information; and
5. a **release program and continuous-integration workflow** that run the recorded comparisons before publication.

The shortest accurate summary is:

Lean checks the formal proofs. Mathematicians review what those proofs mean and how they may be described. The release machinery checks that the reviewed relationships remain intact before publication.

2 The architecture in one picture

Figure 1 names the real components. A solid arrow denotes work performed by a program. A dashed arrow denotes a human decision. The human step is deliberately visible because no program in this repository decides whether arbitrary English has the same meaning as a Lean proposition.

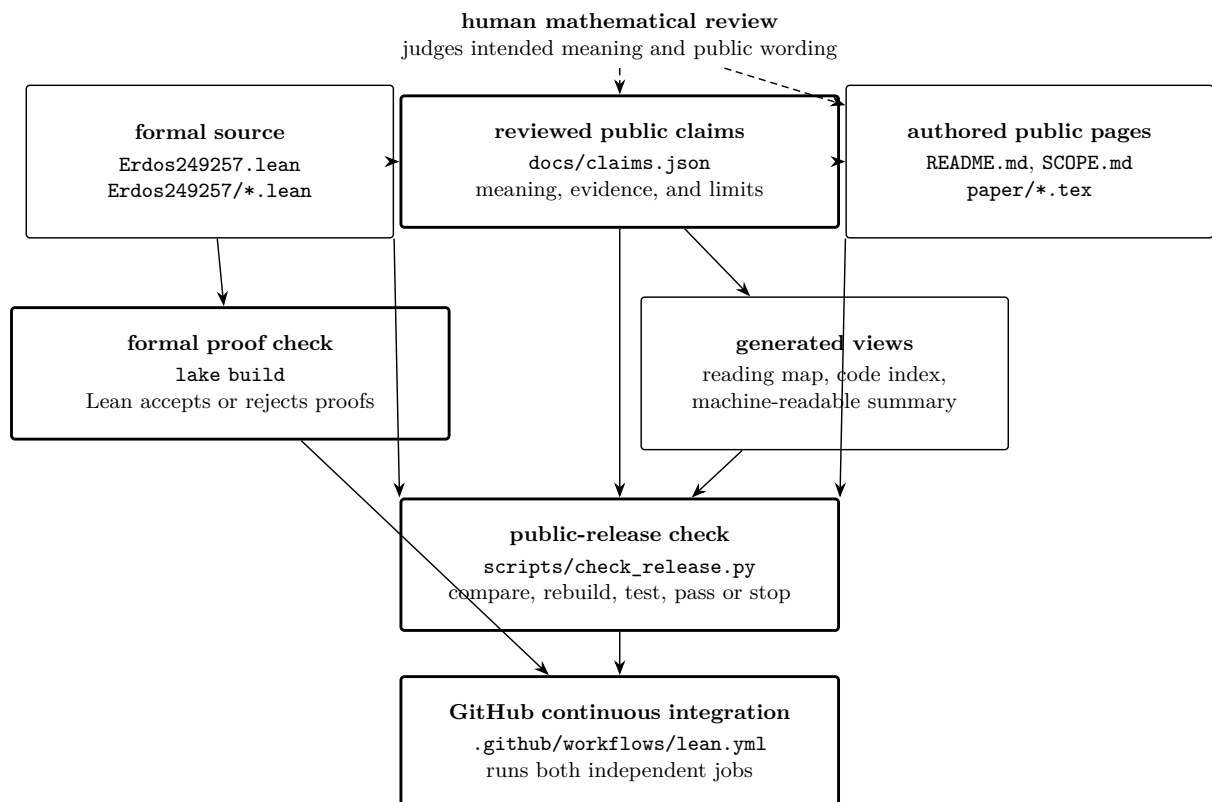


Figure 1: The complete public-release flow. Lean checks formal proofs. A person reviews the bridge from a formal statement to a public claim. The release program checks only relationships that have been written down. The GitHub workflow runs the Lean build and the release checks; it does not add a new source of mathematical authority.

Two commands answer different questions:

Command	Question answered	Question not answered
<code>lake build</code>	Does Lean accept the imported formal statements and proofs?	Does the public explanation describe the intended mathematics correctly?
<code>python3 scripts/check_release.py</code>	Do the recorded claims, links, generated files, paper identities, and selected wording checks agree?	Is every unrecorded sentence true, or is the list of checks complete?

After a mathematical change, both commands are required. The release program confirms the identity of the public Lean source but does not itself run Lean.

3 The real files and their responsibilities

The following tree is intentionally short. It shows the entry points and ownership boundaries, not every one of the hundreds of Lean modules.

<code>Erdos249257.lean</code>	supported root of the Lean library
<code>Erdos249257/</code>	definitions, theorems, finite certificates
<code>examples/</code>	small downstream user of the library
<code>docs/claims.json</code>	reviewed mathematical claims and limits
<code>docs/methodology.json</code>	review rules when a claim changes
<code>docs/publication_contract.json</code>	inventory and identity of shipped papers
<code>docs/publication_evidence.json</code>	historical evidence for checker testing
<code>README.md</code> and <code>SCOPE.md</code>	short public explanation and open boundary
<code>paper/</code>	authored paper source and PDF build rules
<code>docs/SOURCE_MAP.md</code>	routes from questions to Lean files
<code>docs/ORIENTATION.md</code>	generated human reading map
<code>docs/declaration_atlas.json</code>	generated index of Lean declarations
<code>docs/publication_entry_packet.json</code>	generated bounded machine entry point
<code>scripts/</code>	builders, queries, tests, release checks
<code>.github/workflows/lean.yml</code>	checks run on pushes and pull requests

There is no single file that is authoritative for every question. The division is as follows.

Question	Source of truth	What that source controls
What has been checked formally?	<code>Erdos249257.lean</code> and the Lean modules it imports	Formal statements and proofs at the fixed source version named in <code>docs/claims.json</code> .
What may the project say publicly about a result?	<code>docs/claims.json</code>	Public wording, result kind, supporting Lean names, bounded ranges, and the exact questions that remain open.
What review is required when meaning changes?	<code>docs/methodology.json</code>	Change classes, required evidence, and review policy.
Which papers are shipped?	<code>docs/publication_contract.json</code>	Manuscript roles, source and PDF identities, and validation commands.
What supports the historical checker example?	<code>docs/publication_evidence.json</code>	The retained experiment record and its limitations.
What do readers see?	<code>README.md</code> , <code>SCOPE.md</code> , and <code>paper/*.tex</code>	Human explanation, constrained by the reviewed claims but not proof authority themselves.

Table 1: The main ownership boundaries. The three JSON files in the middle have different jobs: mathematical claims, paper inventory, and historical evidence.

Some boundaries run through a file rather than around it. For example, most of the README is authored prose, while marked blocks inside it are written by a builder. The claim file is reviewed data, while scripts refresh source-line coordinates and a module graph inside it. The rule is therefore not “JSON is authored” or “Markdown is generated.” Each generated region names its builder. A stale generated region is rebuilt by that owner, not repaired by hand.

The generated views do not create new mathematics. They select and reorganise information so that a person or another program can enter the repository without reading every source file. Examples include:

- `scripts/build_corpus_descriptor.py`, which writes the human and machine orientation views;
- `scripts/build_declaration_atlas.py`, which writes an index of Lean declarations; and
- `scripts/build_publication_entry_packet.py`, which writes a bounded machine-readable entry point from authored publication metadata.

4 One result from Lean proof to public page

The architecture becomes concrete when one result is followed through every layer. We begin with its public meaning, not its internal name:

Lean has checked a finite calculation at 28 listed input values. The open theorem requires successful values beyond every fixed cutoff. The 28 calculations therefore do not prove the open theorem.

This distinction needs no specialist vocabulary. If a finite list ends at 64, one may choose a cutoff larger than 64. The open theorem still asks for a successful value after that cutoff.

4.1 The formal evidence

The module [Erdos249257/DiagonalPincerCertificatesT64.lean](#) lists the 28 values and contains a Lean theorem assembling their checked calculations. The supported root file [Erdos249257.lean](#) imports that module. Running `lake build` asks Lean to check the theorem and every proof on which it depends.

4.2 The reviewed public record

The corresponding object in `docs/claims.json` contains a public statement, a result kind, the exact finite domain, five supporting Lean declarations, and a link to the still-open requirement. The following is a shortened view of the real entry; ellipses omit repeated paths and values only to fit the page.

```
{
  "id": "certified_kill_instances",
  "status": "verified finite instance",
  "statement": "Kernel-checked ... at 28 explicit ... scales
               through t = 64 ...",
  "bounded_domain": "the 28 values listed in this record",
  "remaining_open_proposition_ids": [
    "remaining_open.unbounded_certificate_supply"
  ],
  "declarations": [
    "Erdos249257/TotientTailPeriodKiller.lean:404",
    "Erdos249257/LcmDiagonalReduction.lean:237",
    "... three further supporting declarations ..."
  ]
}
```

The field called `status` is not a score. Here “verified finite instance” means only that Lean checked a stated finite set of inputs. The field called `id` is an internal lookup handle for software. It appears after the ordinary-language claim because readers should not have to decode it to understand the result.

4.3 The public pages and generated routes

The README and the main mathematical paper state the finite result and the open boundary. Generated orientation files provide links and source locations. A reader or tool can request the same trace with

```
python3 scripts/query_corpus.py --claim certified_kill_instances
```

The release program then checks several concrete relationships: the five Lean names occur in the fixed formal source; the source locations remain nearby; the finite domain is recorded; required public limitation text remains present; and generated views match a fresh rebuild. Lean checks the theorem. The release program checks the recorded description around it. A person is still responsible for judging that the description is mathematically faithful.

5 What happens when a file changes

For a change to a formal statement or proof, the release path is:

1. Edit the relevant file under `Erdos249257/`.
2. Run `lake build`. If Lean rejects a proof, stop and repair the formal source.
3. Review whether the statement, assumptions, intended meaning, or public significance changed. This is a mathematical judgement.
4. If public meaning changed, update `docs/claims.json` and follow the review requirements in `docs/methodology.json`.
5. Update the authored README, scope statement, or paper where a reader encounters the claim.
6. Run the named builders for generated indexes, source coordinates, and paper metadata.
7. Run `python3 scripts/check_release.py`. Any failed recorded relationship stops the release.
8. Push the change. The workflow in `.github/workflows/lean.yml` runs the formal and public-release jobs again in a clean environment.

If only explanatory prose changes, the Lean proof may be untouched, but the public boundary still matters. The release check must run. If the words now make a stronger or different mathematical claim, the claim record and human review must change as well.

If a generated file is stale, its builder should be run. Hand-editing the output can make the worktree appear fixed while leaving the next rebuild ready to erase the change. Builders and generated regions therefore state their ownership explicitly.

5.1 What GitHub runs

The public workflow has two independent jobs.

- The **Lean job** checks the dependency lock, installs the pinned Lean and Mathlib environment, builds the formal library with a memory-bounded planner, completes an ordinary Lake authority check, and builds a small downstream example.
- The **release-surface job** runs the newcomer checks and the public release program. It then verifies source identity, scope, manuscript and licence records, query routes, and deliberately invalid examples for the checking programs.

Continuous integration repeats commands in a clean checkout. It does not perform independent mathematical review and does not turn a consistency check into proof authority.

6 Who decides what

The architecture separates four roles because combining them would make the guarantee sound stronger than it is.

Layer	What it establishes	What it cannot establish
Lean kernel	A formal proof has the written formal statement under its imported definitions, theorems, and assumptions.	That the author chose the intended statement or that an English summary is faithful.
Human mathematical review	The formal statement is judged to have the intended meaning; a selected public description and its limits are judged appropriate.	That Lean accepts the proof, or that every important public claim has been noticed and recorded.
Release program	Recorded names, fields, links, generated files, paper identities, and selected phrases agree with the declared rules.	The meaning of arbitrary unrecorded prose, or completeness of the chosen comparisons.
GitHub workflow	Required commands ran in a clean environment and their processes succeeded.	Independent approval, mathematical truth beyond the formal statements, or quality of human review.

Table 2: The guarantee and non-guarantee of each layer.

The repository records review requirements, but it does not technically force a second independent mathematician to approve every change. A maintainer can edit a Lean statement, its public record, and the prose together. The checks can show that those files agree with one another; they cannot show that the maintainer’s shared decision was correct. Review policy and social process remain outside the automatic trust base.

Similarly, a program-produced fingerprint can prove that two files differ or match byte for byte. It cannot determine which differing file expresses the right mathematics. The architecture uses automation for repeatable comparisons and keeps semantic decisions visible as human work.

7 A claim the checker did not yet cover

The boundary in Table 2 was exposed by one deliberately altered temporary copy of the repository. The real README said that 28 checked finite cases do not provide successful cases beyond every fixed cutoff. In the test copy, that wording was changed to say that the finite cases completed the open requirement.

The formal proofs did not change. The public-release program also passed, because nobody had yet listed that particular claim among the phrases and relationships it must preserve. The error was outside the written checklist.

The repair had two parts. First, the boundary was added to the reviewed public record and required on the appropriate public page. Second, a small opposing test case was added: a temporary copy with the required phrase removed must be rejected. After the repair, the intact text passed and the same false edit failed.

This example demonstrates a coverage boundary, not a reliability score. It does not show how likely the program is to catch a future error. It shows something narrower and more useful for understanding the architecture:

The program can preserve a relationship only after a person has identified and recorded that relationship.

The detailed historical record is kept separately in `docs/publication_evidence.json`. That file records the deliberately altered copies, the original study’s limitations, and the fact that original run logs were not retained. Internal order labels used by the reconstruction scripts

are bookkeeping for those scripts, not concepts a reader needs in order to understand this repository.

8 Operating and adapting the repository

8.1 Where a reader should start

- For the purpose, file map, and trust boundary, read the root `ARCHITECTURE.md` or this paper.
- For the mathematical headline and its exact limits, read `README.md` and `SCOPE.md`.
- For the mathematical argument, read `erdos249-257-main-paper.pdf`.
- To follow one result into Lean, use `docs/SOURCE_MAP.md` or `scripts/query_corpus.py`.
- To change the repository, read `AGENTS.md` and `CONTRIBUTING.md`, then run the checks owned by the changed files.

The public checkout is self-contained. It is not an interface to a private agent system, and no public mathematical claim depends on private files.

8.2 Build dependencies

The file `lean-toolchain` pins the Lean version. The Lake files pin Mathlib and describe the Lean library. Python release tools use the standard library except for named metadata validators installed by continuous integration. The paper Makefile uses Tectonic when available and otherwise uses a standard $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ installation.

The principal local commands are:

```
# formal proofs
lake build

# newcomer-facing architecture and entry surfaces
python3 scripts/check_architecture_guide.py
python3 scripts/check_cold_clone_comprehension.py --quick

# public claims, documents, generated files, and release rules
python3 scripts/check_release.py

# papers
make -C paper
```

Each failing command names the file or relationship that did not meet its recorded rule. A failure is a stop signal, not evidence that the mathematical theorem itself is false. The responsible layer must be inspected: Lean source, reviewed claim, authored document, generated output, or checking code.

8.3 What another project could reuse

Another formal project could reuse the pattern without copying the project's internal names. It would need to:

1. identify the public claims whose strength matters;

2. record, for each selected claim, its ordinary-language meaning, formal evidence, finite domain or assumptions, and unresolved conclusion;
3. define which document is authoritative for proofs, public status, authored explanation, generated views, and release rules;
4. write comparisons for the relationships that should survive routine edits;
5. test each important comparison with both an agreeing and a deliberately disagreeing example; and
6. stop publication when any required proof or release check fails.

The data shapes and scripts in this repository are project-local. The reusable idea is the separation of authority and the explicit bridge from formal result to public claim.

9 Scope and limitations

This paper documents a public release-engineering subsystem for a Lean repository. It is not an architecture for AI agents, model training, or private research orchestration. AI systems contributed to development, but their output has no special authority. Formal proof authority comes from the pinned Lean source checked by Lean; public meaning still requires human review.

The architecture does not establish any of the following:

- a solution of Erdős Problem 249 or the universal form of Problem 257;
- that a formal proposition is the proposition the author intended;
- that every public sentence has been understood by software;
- that every important claim has been entered in the reviewed record;
- that one maintainer’s review is independent or adequate;
- that this project-local design transfers unchanged to another project; or
- that a large number of passing comparisons measures reader comprehension, mathematical coverage, or future error-detection probability.

The architecture does establish a narrower operational fact: before a public release, Lean can check the formal proofs, and software can recheck the specific relationships that people have chosen to record between those proofs and their public descriptions.

10 Related systems

Tools such as `leanblueprint` and `LeanArchitect` connect proof plans or exposition to Lean declarations [1, 2]. Lean Atlas helps people inspect whether a formal proposition states the intended mathematics [3]. Isabelle/DOF assigns checked structure to parts of formal documents [4]. Documentation-consistency and mutation testing research studies how executable systems can be checked against written descriptions or deliberately faulty inputs [5, 6, 7].

This repository combines narrower, ordinary mechanisms: a reviewed JSON record, links to fixed Lean declarations, generated navigation, exact file comparisons, known-answer tests, and a blocking release command. Its purpose is not to interpret unrestricted prose. It is to keep selected, reviewed relationships from drifting silently.

A A small glossary

Lean A proof assistant that checks formal statements and proofs.

Declaration Lean’s name for a definition, theorem, or other named formal object. Declaration names are lookup handles, not reader-facing concepts.

Claim record A structured entry describing one selected public claim, its evidence, its limits, and any unresolved conclusion.

Generated view A file or marked file region written by a program from a more authoritative source. It is an index or presentation, not new proof.

Commit A fixed saved version of the repository. Formal links use a fixed commit so that later edits do not change the theorem being cited.

Digest A short fingerprint computed from a file’s exact bytes. Equal digests support exact identity; they do not judge mathematical meaning.

Continuous integration Commands run automatically in a clean GitHub environment after a push or pull request.

B Reproducibility pointers

The reviewed mathematical claims are in `docs/claims.json`. The paper inventory and file identities are in `docs/publication_contract.json`. The separate historical checker record is in `docs/publication_evidence.json`; executable reconstructions of its deliberately altered copies are described by `experiments/publication_mutations.json` and checked with

```
python3 scripts/run_publication_mutations.py --verify-operators
```

The original raw outputs for that historical exercise are not registered, and the reconstruction does not turn replacement targets into original evidence. The exact formal source cited by the mathematical papers is the saved version whose full identifier is `a16aa24af5236b6508a34b8a4c4b41fdcf620a98`. These identifiers support reproduction; they are not concepts needed for first contact.

References

- [1] P. Massot, *leanblueprint: a plasTeX plugin for formalization blueprints*, [project repository](#), accessed July 2026.
- [2] T. Zhu, P. Monticone, J. Avigad, and S. Welleck, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, 2026, [arXiv:2601.22554](#).
- [3] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv:2604.16347](#).
- [4] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, 2019, pp. 275–292.
- [5] T. Kiecker et al., *CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation*, 2026, [arXiv:2604.19400](#).
- [6] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: Help for the practicing programmer*, *IEEE Computer* 11(4), 1978, pp. 34–41.
- [7] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, *IEEE Transactions on Software Engineering* 37(5), 2011, pp. 649–678.